

```

from collections import deque

def solve_campus_traversal():
    grid = [
        ['S', '.', 'R'],
        ['.', '#', 'D'],
        ['.', '.', 'E']
    ]

    rows = len(grid)
    cols = len(grid[0])

    start_pos = None
    end_pos = None

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 'S':
                start_pos = (r, c)
            elif grid[r][c] == 'E':
                end_pos = (r, c)

    if not start_pos or not end_pos:
        print("Start or End point not found.")
        return

    queue = deque([start_pos])
    visited = set([start_pos])
    parent = {start_pos: None}
    traversal_order = []

    path_found = False

    all_moves = {
        'U': (-1, 0),
        'D': (1, 0),
        'L': (0, -1),
        'R': (0, 1)
    }

    print("Running BFS on the Campus Grid...\n")

    while queue:
        curr_r, curr_c = queue.popleft()
        traversal_order.append((curr_r, curr_c))

        if (curr_r, curr_c) == end_pos:
            path_found = True
            break

        current_symbol = grid[curr_r][curr_c]

        allowed_moves = []

        if current_symbol in ['S', '.']:
            allowed_moves = list(all_moves.values())
        elif current_symbol in all_moves:
            allowed_moves = [all_moves[current_symbol]]

        for dr, dc in allowed_moves:
            next_r, next_c = curr_r + dr, curr_c + dc

            if 0 <= next_r < rows and 0 <= next_c < cols:
                if grid[next_r][next_c] != '#' and (next_r, next_c) not in visited:
                    visited.add((next_r, next_c))
                    parent[(next_r, next_c)] = (curr_r, curr_c)
                    queue.append((next_r, next_c))

    if path_found:
        path = []
        curr = end_pos
        while curr is not None:
            path.append(curr)
            curr = parent[curr]
        path.reverse()

        print("Traversal Order:")
        print(traversal_order)
        print("\nShortest Path:")
        print(path)
        print(f"\nMinimum steps: {len(path) - 1}")

```

```
        print("Minimum steps: ", len(path) - 1, "  
    else:  
        print("-1")  
  
solve_campus_traversal()
```

Running BFS on the Campus Grid...

Traversal Order:

[(0, 0), (1, 0), (0, 1), (2, 0), (0, 2), (2, 1), (2, 2)]

Shortest Path:

[(0, 0), (1, 0), (2, 0), (2, 1), (2, 2)]

Minimum steps: 4